

Predicting Electric Fleet Suitability for U.S. Businesses

Final Project

Dan Morrow

DSC630 Predictive Analytics

Bellevue University

August 2026

Predicting Electric Fleet Suitability for U.S. Businesses

Introduction and Problem Statement

For my DSC630 predictive analytics project, I'm focusing on electric vehicle fleet suitability for U.S. businesses. Many companies are interested in electrifying part of their fleet, but not every vehicle or fleet type is equally ready for that transition. Some vehicles have moderate daily mileage, lower-speed operation, regular stop activity, and route patterns that fit electrification more naturally. Other vehicles have longer daily routes, higher-speed operation, or use patterns that require more attention to range, charging, and downtime.

The business problem is the prioritization of electrification of vehicles within a fleet. Rather than asking whether an entire fleet should go electric at once, the better question is which vehicles should be considered first. The goal of this project is to create an operational EV suitability model that identifies vehicle-days and operating patterns that are stronger candidates for electrification. The project is intended for fleet managers, business leaders, and EV manufacturers who need a practical way to decide where EV planning should begin.

It should be noted that this project focuses on operational EV suitability using the NREL Fleet DNA dataset. Financial factors such as purchase cost, charging infrastructure, incentives, maintenance savings, and payback are still very important when considering electrification, but they aren't modeled here. Instead, the analysis is designed to identify which vehicles should be reviewed first before those additional factors are considered.

Overview of Data Used

The primary dataset for this project is the National Renewable Energy Laboratory's Fleet DNA dataset. Fleet DNA contains commercial fleet vehicle operating data and is designed to

help manufacturers, researchers, and fleet managers understand real-world vehicle behavior.

That makes it useful for this project because EV suitability depends heavily on how vehicles are actually used each day.

The dataset used in the analysis contains 4,705 rows and 372 columns. Each row represents a vehicle-day, or one day of operation for one vehicle. The data includes 486 unique vehicles, 57 deployments, 20 providers, 7 vehicle classes, 11 vocations, 9 vehicle types, 4 drivetrain categories, and 3 fuel categories. Available fields include daily distance, trip count, driving duration, speed data duration, average speed, maximum speed, zero-speed percentage, total stops, stops per mile, stop duration, and vehicle category fields.

Table 1

Fleet DNA Dataset Summary

Measure	Result
Rows	4,705 vehicle-days
Columns	372
Unique vehicles	486
Deployments	57
Providers	20
Vehicle classes	7
Vocations	11
Vehicle types	9
Drivetrain categories	4
Fuel categories	3

Preliminary Analysis Summary

The preliminary analysis confirmed that Fleet DNA supports the main project question with a realistic operational scope. The dataset can show how far vehicles drive, how often they stop, how long they operate, how fast they travel, and what vehicle category they belong to. It can't directly answer every cost, charging installation, or payback question by itself, which is why those factors remain outside the predictive target.

Daily mileage varies widely across the vehicle-days. Average daily distance is about 70.05 miles, while the median is 56.66 miles. The lowest observed daily distance in the selected modeling fields is about 5.13 miles, while the maximum is 568.27 miles. This range is useful because it confirms that the dataset contains very different operating patterns rather than one uniform fleet profile.

The selected data also shows meaningful route behavior. The average vehicle-day includes about 8.81 trips, 2.60 hours of driving time, 109.47 stops, and 43.18% zero-speed time. These values show that a large portion of the dataset includes local or stop-and-go operation, which is central to the EV suitability business problem.

The exploratory results don't establish EV suitability on their own, but they show that the operational variables contain enough variation to build a classification model. Distance, speed, stop activity, trip frequency, and vehicle category will be evaluated together rather than relying on any single measurement.

Data Preparation Process

The first preparation step was loading and inspecting the Fleet DNA composite dataset in Python. The original data is very wide, with 372 columns. Not every available field is needed and would make the model difficult to explain and would add variables that don't directly support the business question, so the analysis narrowed the data to operational and category fields that connect directly to EV suitability.

The missing-value review showed that most of the dataset is complete. Out of 372 columns, 342 have no missing values. Several specialized speed-category fields are missing much of their data, so they were excluded from the modeling dataset. The final focused modeling

dataset contains 4,705 rows and 19 columns, including four identifier fields and 15 operational or category fields. The selected modeling fields have no missing values before the additional stop-duration cleaning step.

The selected operational variables include total distance, trip count, driving duration, speed data duration, driving average speed, maximum speed, percent zero, total stops, stops per mile, and average stop duration. The category variables include vehicle class, vocation, vehicle type, drivetrain, and fuel type. This smaller group keeps the analysis connected to business-friendly concepts that a fleet manager can understand.

One field required additional cleaning before modeling. The average stop duration field contained 261 negative values, which were treated as invalid. In the final code, the negative values were converted to missing values. Extreme high values were capped at the 99th percentile, or 2,846.05 seconds. The remaining missing values were then filled with the median of the cleaned stop-duration field. This cleaned version was used in the predictive models.

Table 2

Selected Data Preparation Results

Measure	Result
Original dataset	4,705 rows × 372 columns
Focused modeling dataset	4,705 rows × 19 columns
Selected fields with missing values	0
Negative stop-duration values cleaned	261
Stop-duration cap	2,846.05 seconds
High Suitability records	3,481 (73.99%)
Not High Suitability records	1,224 (26.01%)

Methods of Analysis

Because Fleet DNA doesn't directly label vehicles as strong or weak EV candidates, I created an engineered EV suitability target based on five operational rules. The rules represent

manageable daily distance, moderate driving time, local-speed behavior, stop-and-go activity, and active route use. Each vehicle-day receives one point for every rule it meets.

Manageable distance is defined as daily mileage between 20 and 120 miles. Moderate driving time is 8 hours or less. Local-speed behavior is an average driving speed of 35 miles per hour or less. Stop-and-go behavior is identified when zero-speed time is at least 30% or stops per mile is at least 1. Active route use is identified when a vehicle-day has at least 4 trips and at least 30 total stops.

Each vehicle-day receives a suitability score from 0 to 5. Records with scores of 4 or 5 are labeled High Suitability, and records with scores from 0 through 3 are labeled Not High Suitability. The final target contains 3,481 High Suitability vehicle-days, or 73.99%, and 1,224 Not High Suitability vehicle-days, or 26.01%. The classes aren't evenly balanced, so the modeling process uses a stratified train-test split and balanced class weights.

The classification model uses ten numeric features: total distance, trip count, driving duration, speed data duration, driving average speed, maximum speed, percent zero, total stops, stops per mile, and cleaned average stop duration. Five categorical fields—vehicle class, vocation, vehicle type, drivetrain, and fuel type—are encoded before modeling.

The data was split into 3,528 training rows and 1,177 test rows. A stratified split preserved the class proportions in both sets. Two models were then built and evaluated: a Decision Tree and a Random Forest. The Decision Tree was chosen because it is easier to explain to a business audience, while the Random Forest was added as a comparison model that combines multiple trees and can provide stronger predictive performance.

Both models were evaluated using accuracy, precision, recall, F1-score, and confusion matrices. Feature importance was also reviewed to identify which operating characteristics

contributed most strongly to model decisions. The final analysis then compared the median operating profile of High Suitability and Not High Suitability vehicle-days to translate the model output into a practical fleet description.

Model Evaluation

The Decision Tree performed well on the test set. It achieved 95.33% accuracy, 99.39% precision, 94.26% recall, and a 96.76% F1-score for the High Suitability class. The confusion matrix shows that the Decision Tree correctly identified 821 High Suitability records and 301 Not High Suitability records. It incorrectly classified 5 Not High records as High and missed 50 High Suitability records.

The Random Forest also performed well and was slightly stronger on several measures. It achieved 95.41% accuracy, 99.04% precision, 94.72% recall, and a 96.83% F1-score. It correctly identified 825 High Suitability records and 298 Not High Suitability records. It incorrectly classified 8 Not High records as High and missed 46 High Suitability records.

Table 3

Model Comparison

Model	Accuracy	Precision	Recall	F1-Score
Decision Tree	0.9533	0.9939	0.9426	0.9676
Random Forest	0.9541	0.9904	0.9472	0.9683

The difference between the two models is small. The Random Forest has slightly higher accuracy, recall, and F1-score, while the Decision Tree has slightly higher precision. The Random Forest correctly identifies four more High Suitability cases than the Decision Tree, while the Decision Tree produces three fewer false-positive High Suitability classifications.

Because the gap is small, the Random Forest isn't dramatically better than the Decision Tree. It is selected as the final predictive model because it provides the strongest overall balance

across the evaluation metrics, especially recall and F1-score. The Decision Tree remains valuable because its performance is nearly identical and its decision structure is easier to communicate to business stakeholders.

Interpretation of Results

The Decision Tree feature importance results identify driving average speed, total stops, and total distance as the three strongest predictors of EV suitability. Driving average speed has the highest importance score at about 0.396, followed by total stops at about 0.310 and total distance at about 0.241. Percent zero and trip count also contribute, but their importance is much smaller than the top three variables.

Table 4

Top Decision Tree Feature Importances

Feature	Importance
driving_average_speed	0.3962
total_stops	0.3101
distance_total	0.2409
percent_zero	0.0287
trip_count	0.0154
voc_id_6	0.0064
stops_per_mile	0.0024

The Random Forest reaches the same overall conclusion. Driving average speed, total stops, and total distance remain the most important predictors, but the Random Forest spreads more importance across additional operational variables such as stops per mile, driving duration, speed data duration, percent zero, and average stop duration. The agreement between the two models is important because it shows that the main business interpretation isn't dependent on only one modeling method.

The results show that EV suitability in this analysis is driven primarily by operating pattern rather than vehicle category alone. Lower average speeds, more stop activity, and manageable daily mileage consistently separate stronger candidates from weaker candidates. Vehicle class, vocation, drivetrain, and fuel type still contribute information, but they don't replace the importance of daily use patterns.

Final Candidate Profile

The final coding analysis adds a direct comparison between High Suitability and Not High Suitability vehicle-days. This step makes the model results easier to translate into a practical screening profile for fleet managers.

Table 5

Median Operating Profile by EV Suitability

Measure	High Suitability	Not High Suitability
Daily distance (miles)	54.39	95.25
Driving duration (hours)	2.25	2.32
Average speed (mph)	23.02	36.58
Percent zero-speed time	47.52%	26.08%
Total stops	92	29
Stops per mile	1.65	0.57
Trip count	8	7

High Suitability vehicle-days have a median daily distance of 54.39 miles compared with 95.25 miles for Not High Suitability vehicle-days. Their median average speed is also lower, at 23.02 mph compared with 36.58 mph. High Suitability vehicle-days spend 47.52% of their time at zero speed, compared with 26.08% for the Not High group.

Stop activity shows an even larger difference. High Suitability vehicle-days have a median of 92 total stops and 1.65 stops per mile. The Not High Suitability group has a median of

29 stops and 0.57 stops per mile. Trip counts are similar at 8 versus 7, and median driving duration is also similar at 2.25 versus 2.32 hours.

Taken together, these results define a clear operational profile. Stronger EV candidates in this dataset tend to have moderate daily mileage, lower-speed local operation, frequent stops, and more stop-and-go activity. That profile matches the feature-importance results from both models and gives fleet managers a straightforward way to interpret what the classification model is identifying.

Final Recommendations

The results support the idea that fleet electrification should be handled as a prioritization problem. Businesses shouldn't assume that every vehicle is equally ready for electrification. Instead, they should use operational data to determine which vehicles deserve a more detailed review first.

Based on the final model and candidate profile, businesses should begin with vehicles that have manageable daily mileage, lower average driving speeds, frequent stop activity, and regular local operation. Vehicles with longer daily mileage or higher-speed operating patterns can still be evaluated, but they should receive additional review of range requirements, charging opportunities, downtime, and route reliability before they are prioritized.

The Random Forest is the final predictive model because it has the best overall balance of accuracy, recall, and F1-score, but the difference from the Decision Tree is small. For business communication, the Decision Tree remains useful because it shows the same major operational drivers in a simpler form. Presenting both models gives the project a stronger balance between predictive performance and interpretability.

The practical recommendation is to use the model as an initial screening tool rather than a final purchase decision. A fleet manager can identify a smaller group of operationally strong candidates and then complete a second review of charging infrastructure, replacement cost, electricity rates, incentives, maintenance savings, driver needs, route reliability, and payback. The model answers the first question in that process: which vehicles should we look at first?

Ethical Considerations

The main ethical concern is that the model can oversimplify a complex business decision if it is treated as more definitive than it is. Fleet electrification depends on factors that aren't fully included in the current model, such as purchase price, charger installation cost, electrical capacity, route reliability, electricity rates, maintenance needs, incentives, and charging downtime. Using the model without those additional reviews could lead to a technically strong classification that doesn't make sense for a specific business.

Another ethical concern is the engineered target variable. Fleet DNA doesn't directly label EV suitability, so the target is based on operational rules created for this project. Those rules are reasonable and transparent, but they still reflect analyst assumptions. Different mileage, speed, stop, or route thresholds would create a different target. The final project therefore treats the label as a decision-support construct rather than an objective measure of real-world EV success.

There is also a fairness concern. Larger businesses and urban fleets often have more access to capital, charging infrastructure, and replacement options than smaller businesses or rural fleets. Operational suitability shouldn't be used to exclude lower-resource organizations

from investment or support. The screening model should identify promising operating patterns, while financial and infrastructure barriers are reviewed separately.

Environmental claims also need to stay within the scope of the analysis. EVs can reduce tailpipe emissions, but total environmental impact depends on electricity generation, battery production, vehicle replacement timing, and how the vehicle is used. This project doesn't model full life-cycle emissions, so it doesn't claim that electrification is automatically the best environmental choice in every situation.

For these reasons, the final model is best used as a decision-support tool. It narrows the field of candidates and provides evidence about operating patterns, but it doesn't replace a complete operational, financial, infrastructure, and environmental review.

Conclusion

This project focused on a practical fleet electrification question: which vehicles should be reviewed first? Using NREL Fleet DNA data, the analysis narrowed a large operational dataset to a focused group of business-friendly variables, created an EV suitability target, and evaluated Decision Tree and Random Forest classification models.

Both models performed strongly, with accuracy above 95%. The Random Forest was selected as the final predictive model because it has slightly higher accuracy, recall, and F1-score. The difference from the Decision Tree is small, and the Decision Tree remains valuable because its results are easier to explain. The agreement between the two models strengthens the final interpretation.

The most important finding is that operational EV suitability is driven mainly by average speed, stop activity, and daily distance. The final candidate profile confirms the same pattern:

High Suitability vehicle-days travel fewer miles, operate at lower speeds, spend more time at zero speed, and make substantially more stops than Not High Suitability vehicle-days.

The final recommendation is to use the model to prioritize vehicles for deeper review rather than to make automatic replacement decisions. Vehicles with moderate mileage, local-speed operation, and frequent stop-and-go activity should be reviewed first for charging needs, cost, incentives, maintenance, range, and payback. This approach gives businesses a practical starting point for electrification without treating every vehicle or fleet as the same.

References

National Renewable Energy Laboratory. (2016). Fleet DNA: Commercial fleet vehicle operating data. NREL Data Catalog. <https://data.nrel.gov/submissions/42>

- Final Project Appendix
- Markdown File

Predicting Electric Fleet Suitability for U.S. Businesses

This notebook uses the NREL Fleet DNA dataset to build and compare classification models for EV suitability. The analysis focuses on daily distance, speed, driving time, stop activity, and vehicle characteristics.

The goal is to identify operating patterns that can help fleet managers decide which vehicles should receive a deeper electrification review.

This is a supplement to the Write up and Presentation and contains the top level information to support those mediums.

Step 1: Load and Inspect the Fleet DNA Data

The Fleet DNA dataset is loaded and reviewed for overall structure.

Output: The dataset contains 4,705 vehicle-day records and 372 columns. A small preview and data type summary are shown below.

```
In [19]: #Import libraries
from pathlib import Path
import pandas as pd
import numpy as np

#Find Fleet DNA file
data_files = (
    list(Path(".").glob("*.csv")) +
    list(Path(".").glob("*.xlsx")) +
    list(Path(".").glob("*.xls"))
)

file_path = next(
    (
        file for file in data_files
        if "fleet" in file.name.lower() or "dna" in file.name.lower()
    ),
    None
)

if file_path is None:
    raise FileNotFoundError()
```

```
        "No Fleet DNA file was found. Put the Fleet DNA file in this folder."
    )

#Load dataset
if file_path.suffix.lower() == ".csv":
    fleet = pd.read_csv(file_path)
else:
    fleet = pd.read_excel(file_path)

#Output basic dataset structure
print(f"Loaded file: {file_path.name}")
print(f"Dataset shape: {fleet.shape}")

print("\nFirst three rows:")
display(fleet.head(3))

print("\nData type summary:")
display(
    fleet.dtypes.value_counts()
    .rename_axis("Data_Type")
    .reset_index(name="Count")
)
```

Loaded file: data_for_fleet_dna_composite_data.csv
Dataset shape: (4705, 372)

First three rows:

	vid	did	pid	class_id	voc_id	type_id	drive_id	fuel_id	day_id	trip_count	...	s
0	236	25	17	7	10	5	0	1	260	4	...	
1	236	25	17	7	10	5	0	1	262	1	...	
2	236	25	17	7	10	5	0	1	263	5	...	

3 rows x 372 columns

Data type summary:

	Data_Type	Count
0	float64	329
1	int64	41
2	str	2

Step 2: Review Missing Values and Data Quality

Missing values and key category fields are reviewed before selecting variables for modeling.

Output: Most columns are complete, including 342 with no missing values. The dataset also contains 486 vehicles, 57 deployments, and multiple vehicle classes, vocations,

drivetrains, and fuel types.

```
In [20]: #Calculate missing values
missing_summary = pd.DataFrame({
    "Column": fleet.columns,
    "Missing_Count": fleet.isna().sum().values,
    "Missing_Percent": (fleet.isna().mean().values * 100).round(2)
})

#Create missing-value groups
missing_groups = pd.cut(
    missing_summary["Missing_Percent"],
    bins=[-1, 0, 25, 50, 75, 100],
    labels=["0%", "1-25%", "26-50%", "51-75%", "76-100%"]
)

missing_group_summary = (
    missing_groups.value_counts()
    .sort_index()
    .rename_axis("Missing_Percent_Group")
    .reset_index(name="Column_Count")
)

#Output missing-value summary
print("Missing value summary:")
display(missing_group_summary)

#Output five columns with highest missingness
print("\nTop five columns by missing percentage:")
display(
    missing_summary
    .sort_values("Missing_Percent", ascending=False)
    .head(5)
)

#Summarize identifier and category fields
category_cols = [
    "vid", "did", "pid", "class_id",
    "voc_id", "type_id", "drive_id", "fuel_id"
]

category_summary = pd.DataFrame({
    "Column": category_cols,
    "Unique_Values": [
        fleet[col].nunique(dropna=True)
        for col in category_cols
    ]
})

print("\nCategory and ID field summary:")
display(category_summary)
```

Missing value summary:

	Missing_Percent_Group	Column_Count
0	0%	342
1	1-25%	14
2	26-50%	2
3	51-75%	6
4	76-100%	8

Top five columns by missing percentage:

	Column	Missing_Count	Missing_Percent
359	spd_cat_6_std_speed	4705	100.0
369	spd_cat_8_std_speed	4705	100.0
368	spd_cat_8_mean_speed	4705	100.0
364	spd_cat_7_std_speed	4705	100.0
363	spd_cat_7_mean_speed	4705	100.0

Category and ID field summary:

	Column	Unique_Values
0	vid	486
1	did	57
2	pid	20
3	class_id	7
4	voc_id	11
5	type_id	9
6	drive_id	4
7	fuel_id	3

Step 3: Review Candidate Modeling Variables

Candidate variables are reviewed to identify the fields most relevant to EV suitability.

Output: The selected variables cover distance, trips, driving time, speed, stop activity, and vehicle characteristics.

```
In [21]: #Select candidate modeling variables
candidate_model_columns = [
    "distance_total",
    "trip_count",
    "driving_data_duration_hrs",
    "speed_data_duration_hrs",
```

```
    "driving_average_speed",
    "max_speed",
    "percent_zero",
    "total_stops",
    "stops_per_mile",
    "average_stop_duration",
    "class_id",
    "voc_id",
    "type_id",
    "drive_id",
    "fuel_id"
]

#Summarize selected variables
candidate_summary = pd.DataFrame({
    "Variable": candidate_model_columns,
    "Data_Type": [
        str(fleet[col].dtype)
        for col in candidate_model_columns
    ],
    "Missing_Percent": [
        round(fleet[col].isna().mean() * 100, 2)
        for col in candidate_model_columns
    ]
})

print("Selected candidate modeling variables:")
display(candidate_summary)
```

Selected candidate modeling variables:

	Variable	Data_Type	Missing_Percent
0	distance_total	float64	0.0
1	trip_count	int64	0.0
2	driving_data_duration_hrs	float64	0.0
3	speed_data_duration_hrs	float64	0.0
4	driving_average_speed	float64	0.0
5	max_speed	float64	0.0
6	percent_zero	float64	0.0
7	total_stops	float64	0.0
8	stops_per_mile	float64	0.0
9	average_stop_duration	float64	0.0
10	class_id	int64	0.0
11	voc_id	int64	0.0
12	type_id	int64	0.0
13	drive_id	int64	0.0
14	fuel_id	int64	0.0

Step 4: Create a Focused Modeling Dataset

The selected fields are combined into the final modeling dataset.

Output: The focused dataset contains 4,705 rows and 19 columns with no missing values in the selected fields. Average daily distance is 70.05 miles, with 8.81 trips, 2.60 driving hours, and 109.47 stops per vehicle-day.

```
In [22]: #Select identifier columns
id_columns = [
    "vid",
    "did",
    "pid",
    "day_id"
]

#Select modeling columns
model_columns = [
    "distance_total",
    "trip_count",
    "driving_data_duration_hrs",
    "speed_data_duration_hrs",
    "driving_average_speed",
    "max_speed",
```

```

    "percent_zero",
    "total_stops",
    "stops_per_mile",
    "average_stop_duration",
    "class_id",
    "voc_id",
    "type_id",
    "drive_id",
    "fuel_id"
]

#Create focused modeling dataset
model_data = fleet[id_columns + model_columns].copy()

#Replace infinite values with missing values
model_data = model_data.replace([np.inf, -np.inf], np.nan)

#Output dataset shape
print("Focused modeling dataset shape:", model_data.shape)

#Preview top records
print("\nFirst three rows:")
display(model_data.head(3))

#Check selected fields for missing values
selected_missing = pd.DataFrame({
    "Column": model_data.columns,
    "Missing_Count": model_data.isna().sum().values
})

print("\nColumns with missing values:")
missing_selected = selected_missing[
    selected_missing["Missing_Count"] > 0
]

if missing_selected.empty:
    print("No missing values found in the selected modeling fields.")
else:
    display(missing_selected)

#Summarize key operational fields
summary_columns = [
    "distance_total",
    "trip_count",
    "driving_data_duration_hrs",
    "driving_average_speed",
    "percent_zero",
    "total_stops",
    "stops_per_mile"
]

print("\nKey operational summary statistics:")
display(
    model_data[summary_columns]
    .describe()
    .T[["mean", "50%", "min", "max"]]

```

```
        .rename(columns={"50%": "median"})
        .round(2)
    )
```

Focused modeling dataset shape: (4705, 19)

First three rows:

	vid	did	pid	day_id	distance_total	trip_count	driving_data_duration_hrs	speed_u
0	236	25	17	260	172.913326	4	7.902778	
1	236	25	17	262	22.292786	1	1.855833	
2	236	25	17	263	87.400215	5	3.146667	

Columns with missing values:
No missing values found in the selected modeling fields.

Key operational summary statistics:

	mean	median	min	max
distance_total	70.05	56.66	5.13	568.27
trip_count	8.81	8.00	1.00	40.00
driving_data_duration_hrs	2.60	2.25	0.18	13.80
driving_average_speed	26.16	24.17	8.45	54.48
percent_zero	43.18	43.78	0.69	95.33
total_stops	109.47	80.00	2.00	825.00
stops_per_mile	2.01	1.45	0.06	16.75

Step 5: Create an EV Suitability Label

A rule-based target is created using mileage, driving time, average speed, stop-and-go activity, and active route use.

Output: Vehicle-days meeting at least 4 of the 5 rules are classified as High Suitability. The final target includes 3,481 High Suitability records (73.99%) and 1,224 Not High Suitability records (26.01%).

```
In [23]: #Output cleaning results
print(
    "Negative average stop duration values cleaned:",
    negative_stop_count
)

print(
    "Average stop duration 99th percentile cap:",
    round(stop_duration_cap, 2)
)
```

```

#Output suitability rule counts
rule_summary = (
    model_ready[score_columns]
    .sum()
    .rename_axis("Rule")
    .reset_index(name="Vehicle_Day_Count")
)

print("\nSuitability rule counts:")
display(rule_summary)

#Output target distribution
target_distribution = (
    model_ready["ev_suitability_label"]
    .value_counts()
    .rename_axis("EV_Suitability_Label")
    .reset_index(name="Count")
)

target_distribution["Percent"] = (
    target_distribution["Count"] /
    len(model_ready) * 100
).round(2)

print("\nEV suitability target distribution:")
display(target_distribution)

```

Negative average stop duration values cleaned: 261
 Average stop duration 99th percentile cap: 2846.05

Suitability rule counts:

	Rule	Vehicle_Day_Count
0	manageable_distance	3681
1	moderate_driving_time	4565
2	local_speed_pattern	3911
3	stop_go_pattern	3965
4	active_route	3358

EV suitability target distribution:

	EV_Suitability_Label	Count	Percent
0	High Suitability	3481	73.99
1	Not High Suitability	1224	26.01

Step 6: Build and Evaluate a Decision Tree Model

The first classification model is a Decision Tree using the selected operating and vehicle characteristics.

Output: The model performs well on the test data, with about 95.3% accuracy. The detailed metrics and confusion matrix are shown below.

```
In [24]: #Create Decision Tree evaluation summary
decision_tree_results = pd.DataFrame({
    "Metric": [
        "Accuracy",
        "Precision",
        "Recall",
        "F1 Score"
    ],
    "Score": [
        accuracy,
        precision,
        recall,
        f1
    ]
})

decision_tree_results["Score"] = (
    decision_tree_results["Score"].round(4)
)

#Output training and test sizes
print("Training rows:", X_train.shape[0])
print("Test rows:", X_test.shape[0])

#Output evaluation metrics
print("\nDecision Tree Evaluation:")
display(decision_tree_results)

#Create confusion matrix
conf_matrix = confusion_matrix(y_test, y_pred)

conf_matrix_df = pd.DataFrame(
    conf_matrix,
    index=["Actual Not High", "Actual High"],
    columns=["Predicted Not High", "Predicted High"]
)

print("\nDecision Tree Confusion Matrix:")
display(conf_matrix_df)
```

Training rows: 3528

Test rows: 1177

Decision Tree Evaluation:

	Metric	Score
0	Accuracy	0.9533
1	Precision	0.9939
2	Recall	0.9426
3	F1 Score	0.9676

Decision Tree Confusion Matrix:

	Predicted Not High	Predicted High
Actual Not High	301	5
Actual High	50	821

Step 7: Interpret the Decision Tree Model

Feature importance is used to identify which variables contribute most to the Decision Tree predictions.

Output: Average speed, total stops, and daily distance are the strongest predictors. The pattern favors lower-speed, stop-heavy, moderate-distance vehicle use.

```
In [26]: #Get Decision Tree feature importance
dt_model = decision_tree_model.named_steps["model"]

dt_features = decision_tree_model.named_steps[
    "preprocessor"
].get_feature_names_out()

importance_df = pd.DataFrame({
    "Feature": dt_features,
    "Importance": dt_model.feature_importances_
})

#Clean feature names
importance_df["Feature"] = importance_df["Feature"].str.replace(
    r"^(cat_|num_)",
    "",
    regex=True
)

#Output top five Decision Tree features
print("Top Decision Tree feature importances:")

display(
    importance_df
    .sort_values("Importance", ascending=False)
    .head(5)
    .round(4)
)
```

Top Decision Tree feature importances:

	Feature	Importance
38	driving_average_speed	0.3962
41	total_stops	0.3101
34	distance_total	0.2409
40	percent_zero	0.0287
35	trip_count	0.0154

Step 8: Build and Evaluate a Random Forest Model

A Random Forest is built as a comparison to the Decision Tree.

Output: The Random Forest performs slightly better overall, with about 95.4% accuracy and the best balance of accuracy, recall, and F1-score.

```
In [25]: #Create Random Forest evaluation summary
random_forest_results = pd.DataFrame({
    "Metric": [
        "Accuracy",
        "Precision",
        "Recall",
        "F1 Score"
    ],
    "Score": [
        rf_accuracy,
        rf_precision,
        rf_recall,
        rf_f1
    ]
})

random_forest_results["Score"] = (
    random_forest_results["Score"].round(4)
)

#Output evaluation metrics
print("Random Forest Evaluation:")
display(random_forest_results)

#Create confusion matrix
rf_conf_matrix = confusion_matrix(y_test, rf_pred)

rf_conf_matrix_df = pd.DataFrame(
    rf_conf_matrix,
    index=["Actual Not High", "Actual High"],
    columns=["Predicted Not High", "Predicted High"]
)
```

```
print("\nRandom Forest Confusion Matrix:")
display(rf_conf_matrix_df)
```

Random Forest Evaluation:

	Metric	Score
0	Accuracy	0.9541
1	Precision	0.9904
2	Recall	0.9472
3	F1 Score	0.9683

Random Forest Confusion Matrix:

	Predicted Not High	Predicted High
Actual Not High	298	8
Actual High	46	825

Step 9: Compare Model Performance

The two models are compared across accuracy, precision, recall, and F1-score.

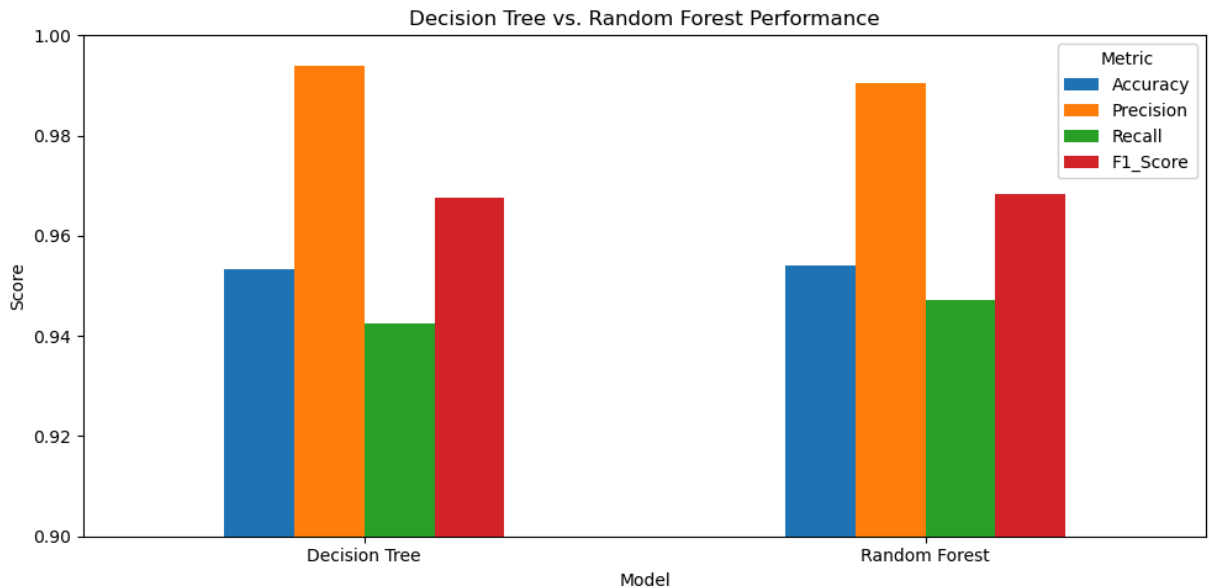
Output: Performance is very similar, but the Random Forest has slightly higher accuracy, recall, and F1-score.

```
In [13]: #Create model comparison chart
comparison_plot = model_comparison.set_index("Model")

comparison_plot.plot(
    kind="bar",
    figsize=(10, 5)
)

plt.title("Decision Tree vs. Random Forest Performance")
plt.ylabel("Score")
plt.xlabel("Model")
plt.ylim(0.90, 1.00)
plt.xticks(rotation=0)
plt.legend(title="Metric")
plt.tight_layout()

#Output model comparison
plt.show()
```



Step 10: Interpret Random Forest Feature Importance

Feature importance is reviewed for the Random Forest.

Output: Average speed, total stops, and daily distance are among the strongest predictors, with additional importance spread across several operating variables.

```
In [14]: #Get fitted Random Forest and feature names
rf_model = random_forest_model.named_steps["model"]
rf_features = random_forest_model.named_steps[
    "preprocessor"
].get_feature_names_out()

#Create feature importance dataframe
rf_importance = pd.DataFrame({
    "Feature": rf_features,
    "Importance": rf_model.feature_importances_
})

#Clean feature names
rf_importance["Feature"] = rf_importance["Feature"].str.replace(
    r"^(cat_|num_)", "", regex=True
)

#Sort feature importance
rf_importance = rf_importance.sort_values(
    "Importance", ascending=False
)

#Create Random Forest feature importance chart
top_rf_importance = rf_importance.head(10).sort_values("Importance")

plt.figure(figsize=(9, 5))
plt.barh(
    top_rf_importance["Feature"],
```

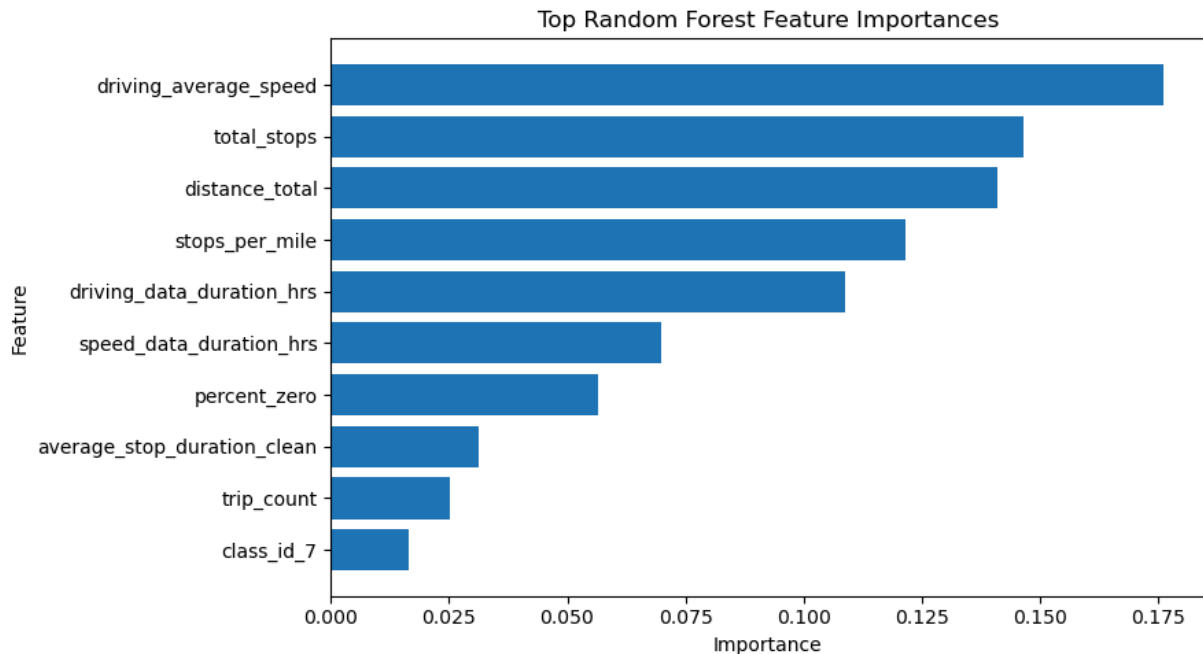
```

top_rf_importance["Importance"]
)

plt.title("Top Random Forest Feature Importances")
plt.xlabel("Importance")
plt.ylabel("Feature")
plt.tight_layout()

#Output feature importance
plt.show()

```



Step 11: Compare Feature Importance Between Models

This step compares the two models.

Output: Both models identify speed, stops, and distance as the main operating factors associated with EV suitability.

```

In [15]: #Get Decision Tree feature importance
dt_model = decision_tree_model.named_steps["model"]
dt_features = decision_tree_model.named_steps[
    "preprocessor"
].get_feature_names_out()

dt_importance = pd.DataFrame({
    "Feature": dt_features,
    "Decision Tree": dt_model.feature_importances_
})

#Clean feature names
dt_importance["Feature"] = dt_importance["Feature"].str.replace(
    r"^(cat_|num_)", "", regex=True
)

```

```
)

#Prepare Random Forest importance
rf_compare = rf_importance[
    ["Feature", "Importance"]
].rename(columns={"Importance": "Random Forest"})

#Combine model importances
feature_comparison = dt_importance.merge(
    rf_compare,
    on="Feature",
    how="outer"
).fillna(0)

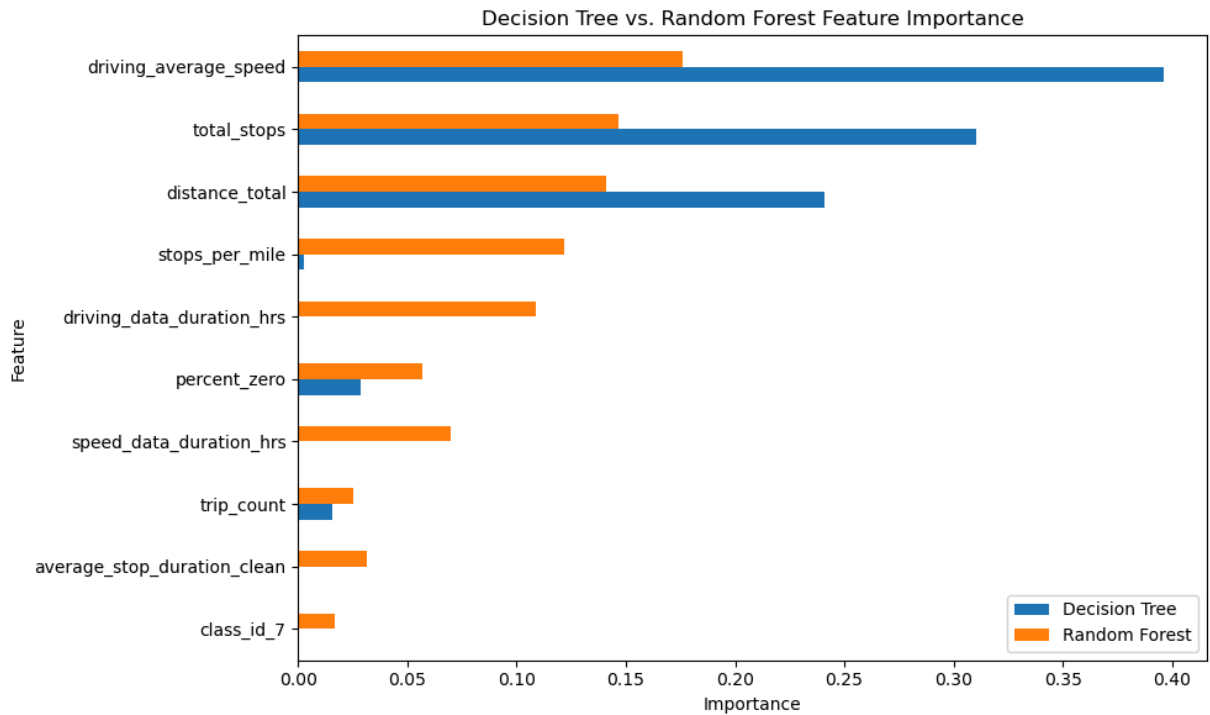
#Select most important features overall
feature_comparison["Combined"] = (
    feature_comparison["Decision Tree"] +
    feature_comparison["Random Forest"]
)

top_features = (
    feature_comparison
    .sort_values("Combined", ascending=False)
    .head(10)
    .sort_values("Combined")
)

#Create comparison chart
top_features.set_index("Feature")[
    ["Decision Tree", "Random Forest"]
].plot(
    kind="barh",
    figsize=(10, 6)
)

plt.title("Decision Tree vs. Random Forest Feature Importance")
plt.xlabel("Importance")
plt.ylabel("Feature")
plt.tight_layout()

#Output feature comparison
plt.show()
```



Step 12: Visualize the Random Forest Confusion Matrix

A confusion matrix visual is created.

Output: Most predictions are correct, with 8 false positives and 46 false negatives.

```
In [17]: #Create Random Forest confusion matrix
rf_conf_matrix = confusion_matrix(y_test, rf_pred)

plt.figure(figsize=(6, 5))
plt.imshow(rf_conf_matrix)

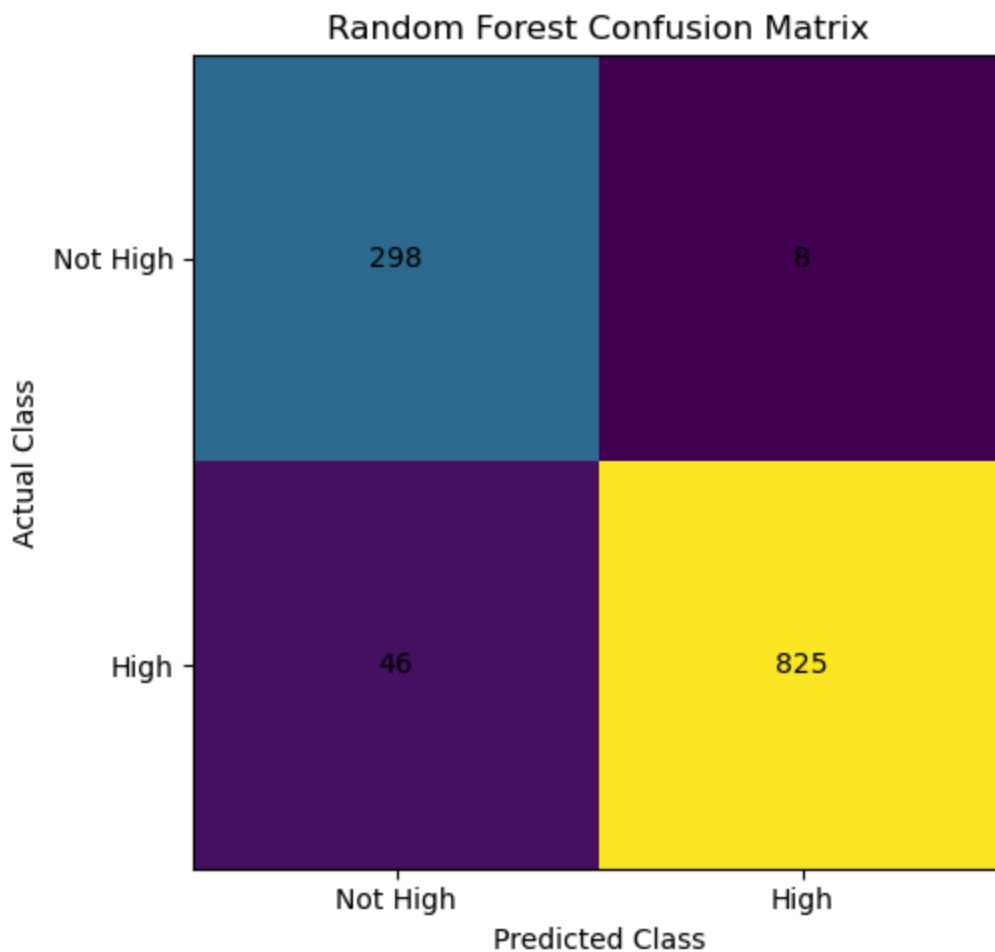
plt.title("Random Forest Confusion Matrix")
plt.xlabel("Predicted Class")
plt.ylabel("Actual Class")

plt.xticks(
    [0, 1],
    ["Not High", "High"]
)

plt.yticks(
    [0, 1],
    ["Not High", "High"]
)

#Add values to matrix
for i in range(rf_conf_matrix.shape[0]):
    for j in range(rf_conf_matrix.shape[1]):
        plt.text(
```

```
j,  
i,  
rf_conf_matrix[i, j],  
ha="center",  
va="center"  
)  
  
plt.tight_layout()  
  
#Output confusion matrix  
plt.show()
```



Step 13: Select the Final Model

The final model is selected based on overall predictive performance and usefulness for the business problem.

Decision: Random Forest is selected because it provides the best overall balance of accuracy, recall, and F1-score.

Step 14: Define a Strong EV Candidate

The final comparison translates the model results into a practical fleet profile.

Output: Stronger candidates generally have moderate daily mileage, lower average speeds, and frequent stop-and-go activity.

```
In [27]: #Compare key operating patterns by suitability
candidate_profile = model_ready.groupby(
    "ev_suitability_label"
)[
    [
        "distance_total",
        "driving_average_speed",
        "percent_zero",
        "total_stops",
        "stops_per_mile"
    ]
].median().round(2)

#Rename columns for cleaner output
candidate_profile.columns = [
    "Daily_Miles",
    "Average_Speed",
    "Percent_Zero",
    "Total_Stops",
    "Stops_Per_Mile"
]

#Output candidate profile
display(candidate_profile)
```

	Daily_Miles	Average_Speed	Percent_Zero	Total_Stops	Stops_Per_
ev_suitability_label					
High Suitability	54.39	23.02	47.52	92.0	
Not High Suitability	95.25	36.58	26.08	29.0	

Conclusion

The model should be used as a screening tool rather than an automatic replacement decision. The suitability target is rule-based and does not include financial, charging, incentive, or payback factors.

Fleet managers should prioritize vehicles with moderate daily mileage, lower-speed operation, and frequent stop activity for a deeper EV review.